Processing and Visualising Large DEMs on a Laptop

Progress Report: Data Preparation and Baseline Benchmarks of Standard Terrain Tools

Team GeoGuessers | Spatial Informatics

Swayam Goyal (2024111012) Siddharth Choudhary (2024111021)
Yakkala Viswanadha Kartikeya (2024111033)

Abstract

We investigate how far large, high-resolution Digital Elevation Models (DEMs) can be processed and visualised on a single laptop, and where the actual bottleneck lies. On one Himalayan study area (Uttarakhand, ∼100 km × 100 km) we prepared real DEMs at 90, 30, 8 and 2 m and synthetic 2 m and 1 m grids (up to 10.8 billion cells, 43 GB as float32, 2.6× the laptop's RAM). With a purpose-built benchmark harness that measures exact per-process time, memory, CPU and I/O under controlled cold- and warm-cache conditions, we ran 832 measured runs of GDAL's `gdaldem` (10 operator/method configurations), seven storage layouts, and QGIS rendering. Three findings stand out. (1) `gdaldem` never runs out of memory: it streams the grid and peaks at 0.87 GB even at 1 m; it only slows down linearly, at 19–25 ns per cell (3.5 min for one slope pass at 1 m). (2) The bottleneck is single-threaded computation (∼73% of run time, 1 of 22 threads used), not disk (14%) or memory; the file-cache state changes run time by ≤2%. (3) For display, the bottleneck is reading full-resolution data: without a pyramid, a whole-area QGIS redraw takes 32 s at 1 m, with one it takes 0.09 s at every resolution. These results motivate the next phase: a tiled, parallel CPU/GPU engine and on-demand computation from pyramid levels.

1 Objectives

Following our discussion, the project is approached as a systems study: given a very large, high-resolution DEM and limited hardware, determine how to process and visualise only the data currently required, using multi-resolution (pyramid) representations, on-the-fly computation and appropriate CPU/GPU processing, while identifying the actual bottleneck rather than assuming one. Our literature review identified two gaps that this project addresses:

G1 No study reports where standard GDAL/QGIS terrain tools stop being practical on a single laptop, separated into stages (disk read, computation, write, display).

G2 No study computes terrain derivatives on demand from pyramid levels, or measures the error of coarse-level slope against the time it saves.

This report covers the work towards G1: data preparation, the benchmark methodology, and the baseline measurements.

2 Data

All experiments use one study area, so that grid size is the only variable that changes (Figure 1, Table 1). The area is HMA tile-571 in the Garhwal Himalaya (Nanda Devi region), with elevations from 670 m to 7,790 m. All grids were reprojected to WGS 84 / UTM 44N (EPSG:32644) on one aligned grid and stored as Cloud-Optimized GeoTIFFs (512-pixel tiles, ZSTD compression, internal overviews forming the pyramid).

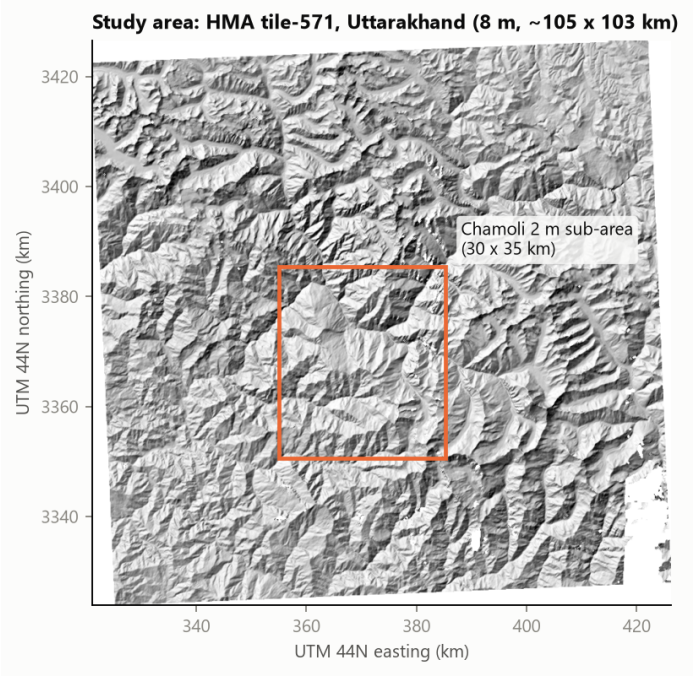


Figure 1: Study area (shaded relief of the 8 m DEM) and the Chamoli sub-area covered by the real 2 m DEM.

Table 1: Resolution ladder. Real grids are used for timing and accuracy; synthetic grids only for timing.

Res.	Source	Type	Cells	Role
90 m	Copernicus GLO-90 (OpenTopography)	real	1.3 M	coarsest level
30 m	Copernicus GLO-30 [1]	real	12 M	baseline level
8 m	NASA High Mountain Asia 8 m mosaic [2]	real	169 M	finest real level, whole area
2 m	Chamoli 2 m composite [3]	real	264 M	accuracy truth (sub-area)
2 m	cubic upsample of the 8 m tile	synthetic	2.7 B	timing only
1 m	cubic upsample of the 8 m tile	synthetic	10.8 B	timing only; 43 GB > 16 GB RAM

Data checks. All downloads were checksum-verified. The 2 m and 8 m DEMs agree well where they overlap (2 m averaged to 8 m minus the 8 m DEM: median −0.7 m, NMAD 5.1 m over 14 million cells). The 8 m DEM sits 24.5 m below Copernicus (ellipsoidal vs geoid heights), which does not affect slope, aspect or hillshade. Since no free 1 m DEM exists for this region, we tested how reliable synthetic grids are: on a 16-million-cell crop, an 8 m grid rebuilt from 16 m gave the same `gdaldem` run time as the real one (0.87–0.90 s) but 2.3–3.0° slope RMSE. Synthetic grids are therefore used for timing only, with cubic resampling (nearest-neighbour data compresses 26% better than real terrain and would distort read times).

3 Benchmark methodology

Machine. Intel Core Ultra 9 185H (6 performance, 8 efficiency and 2 low-power cores; 22 threads), 16 GB RAM, NVMe SSD (cold reads measured at 2.7–3.3 GB/s), NVIDIA RTX 4060 Laptop GPU (8 GiB, not used by these tools), Windows 11, GDAL 3.13.3, QGIS 3.44.14.

Harness. Each measurement runs one tool as a separate process. For every run we record wall time and, read exactly from the finished process, its peak memory (peak working set), CPU time and bytes read/written, plus the bytes physically read from disk. One row per run is stored with the dataset, operator, method, cache state and the code version. Sweeps are resumable.

Protocol.

- **Cold cache:** the Windows file cache (standby list) is emptied before each cold run (Sysinternals RAMMap); we verified that the whole file is then read from disk again. **Warm cache:** immediate repeat.
- **Repeats:** real grids: 5 cold runs and 1 warm-up + 5 warm runs; synthetic 2 m/1 m: 3 cold + 1 warm. Medians are reported.
- **Core pinning:** `gdaldem` measured 1.7× slower on an efficiency core and up to 2.7× on a low-power core, and Windows can move a process between them mid-run (one unpinned run took 108.9 s instead of 56 s). All tools are pinned to the performance cores, i.e. measured at their best.
- **Background activity:** Windows Defender scanning our 10–40 GB temporary outputs held disk space and slowed following runs by 25–35%. Only the two scratch folders were excluded from scanning, and the harness waits until deleted outputs are released before the next run. All final numbers were re-run under these conditions.
- **Stage split:** two reference runs per grid, a pure read + decompress and a copy to an uncompressed GeoTIFF (the same read and same-size write as `gdaldem`), give write ≈ copy − read and compute ≈ `gdaldem` − copy.

Experiments. **E1:** `gdaldem` slope, aspect, hillshade (Horn and Zevenbergen–Thorne [4, 5]), TRI (Riley, Wilson), TPI and roughness on all six grids, cold and warm (560 runs; slope and hillshade only at 1 m). **E2:** seven storage layouts (strips vs tiles, 256 vs 512 tiles, none/LZW/DEFLATE/ZSTD) of the 8 m and 2 m grids: write, read and slope (224 runs). **QGIS display:** open a layer and render one 1920×1080 view, whole area or zoomed in, with or without the pyramid (48 runs).

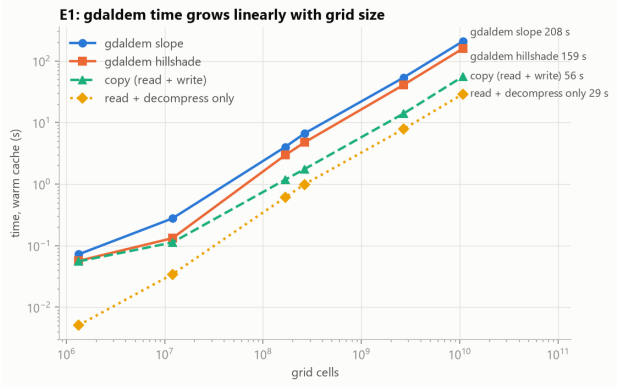
4 Results

4.1 E1: `gdaldem` across grid sizes

Table 2: `gdaldem` (Horn) by grid size. Medians; ns/cell for slope, warm cache.

Grid	Cells	Slope cold	Slope warm	Hillshade	ns/cell	Peak memory
90 m	1.3 M	0.08 s	0.07 s	0.06 s	(start-up)	42 MB
30 m	12 M	0.29 s	0.28 s	0.13 s	23.1	96 MB
8 m	169 M	4.10 s	4.02 s	2.96 s	23.8	745 MB
2 m real	264 M	6.67 s	6.63 s	4.74 s	25.1	868 MB
2 m synthetic	2.7 B	54.2 s	53.7 s	41.0 s	19.9	870 MB
1 m synthetic	10.8 B	208.3 s	208.4 s	158.9 s	19.3	872 MB

- **No memory limit:** `gdaldem` streams the grid and processes 43 GB of data with 0.87 GB.
- **Linear time:** 19–25 ns per cell once start-up stops mattering; 3.5 min for slope at 1 m.
- **Single-threaded:** 0.99 of 22 threads used at every size.
- **Cache does not matter:** cold vs warm differ by ≤2%. At 2 m/1 m even warm runs read 3.8/14.2 GB from disk (at 1 m the file exceeds the cache; at 2 m `gdaldem`'s own ∼11 GB output pushes it out) and take the same time.



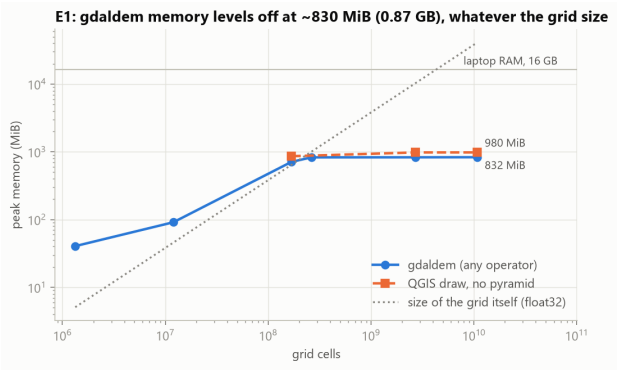
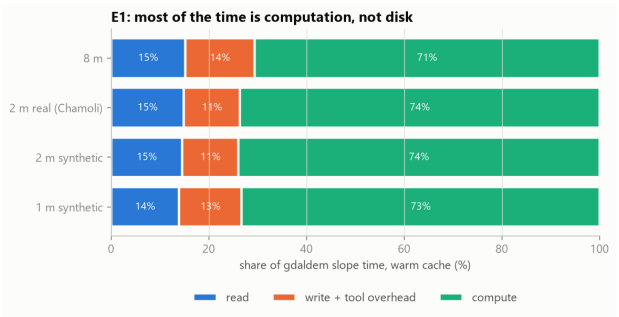


Figure 2: Left: run time grows linearly with the number of cells. Right: peak memory levels off at ∼0.87 GB (GDAL's block cache plus the program), far below the size of the grid, which exceeds the 16 GB RAM at 1 m.



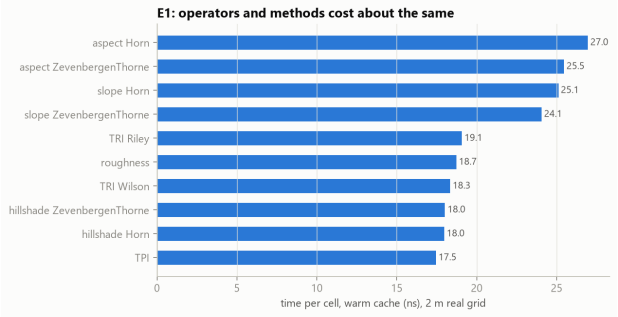


Figure 3: Left: share of slope run time per stage; computation is ∼73% at every size. Right: cost per cell of all ten configurations on the real 2 m grid.

All operators cost 17.5–27 ns per cell; Horn and Zevenbergen–Thorne differ by at most 6% (slope 4%, aspect 6%, hillshade 0%). Hillshade is cheaper than slope partly because it writes 1-byte instead of 4-byte output.

4.2 E2: storage layout

Table 3: Storage layouts of the 2 m synthetic grid (2.7 billion cells). Cold-cache medians.

Layout	File size	Write	Read + decompress	`gdaldem` slope
Strips, uncompressed (GDAL default)	10.8 GB	13.9 s	4.0 s	**47.1 s**
Tiles 256, uncompressed	10.9 GB	13.5 s	4.0 s	51.8 s
Tiles 512, uncompressed	10.9 GB	13.3 s	3.3 s	49.1 s
Tiles 512, LZW	5.0 GB	88.9 s	20.1 s	65.7 s
Tiles 512, DEFLATE	3.9 GB	113.4 s	11.4 s	57.0 s
Tiles 512, ZSTD (our pyramids)	**3.8 GB**	123.9 s	**7.8 s**	53.7 s
Tiles 256, ZSTD	3.8 GB	121.0 s	8.6 s	55.3 s

Layout changes `gdaldem` time by up to 40%, because decompression runs inside the single thread: LZW is 39% slower than uncompressed strips, ZSTD only 14%. ZSTD is the best compressed format (2.9× smaller than uncompressed and decoding 2.6× faster than LZW); LZW, a common default, is the slowest layout and the largest compressed one. Writing compressed files is 6–9× slower, a one-time cost when the pyramid is built. The real 8 m grid shows the same pattern (slope 3.42 s uncompressed, 3.94 s ZSTD, 4.78 s LZW).

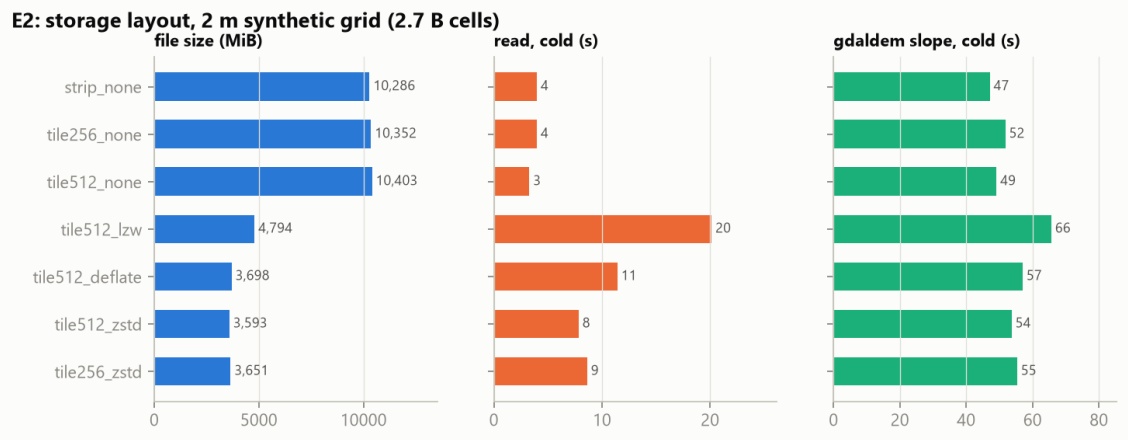


Figure 4: E2: file size, read time and `gdaldem` slope time per storage layout (2 m synthetic grid).

4.3 QGIS display

Table 4: QGIS: time to render one 1920×1080 view (cold cache), and peak memory for the whole-area view.

Grid	Whole area, pyramid	Whole area, no pyramid	Zoomed in (either)	Peak memory (pyramid / none)
8 m	0.09 s	0.79 s	0.10 s	209 / 904 MB
2 m synthetic	0.09 s	8.82 s	0.10 s	210 / 1027 MB
1 m synthetic	0.09 s	**31.9 s**	0.11 s	210 / 1028 MB

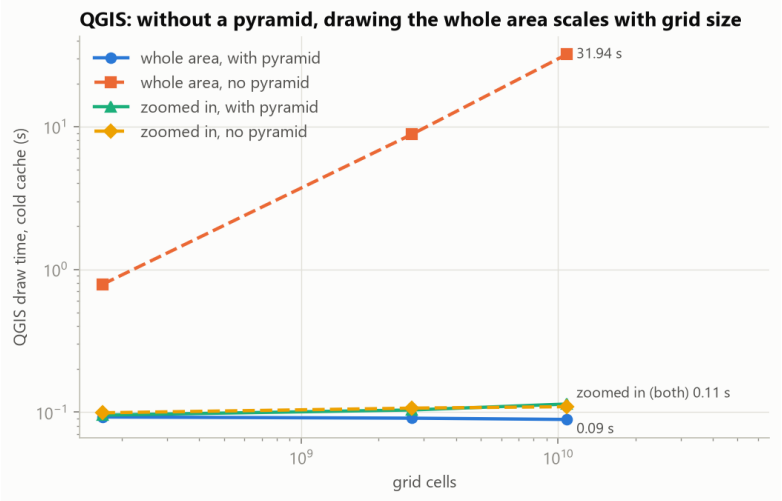


Figure 5: Without a pyramid, a whole-area redraw reads the full-resolution grid and grows with its size; with the pyramid, and for zoomed-in views, the cost stays constant.

Opening a layer takes 0.02–0.03 s in all cases. Only zoomed-out views need the pyramid: a zoomed-in view reads just its window. Without a pyramid, each whole-area redraw of the 1 m grid reads its whole full-resolution level, about 12.7 GB (360× slower than with the pyramid), which makes interactive use impossible.

5 Discussion

Where standard tools stop being practical (G1). On a 16 GB laptop with an NVMe SSD the standard tools do not fail at 1 m; they stop being practical. Processing is limited by single-threaded

computation, with disk and memory far from their limits. Display is limited by reading full-resolution data when no pyramid is used.

Comparison with the literature and our pilot. The literature generally reports data movement as the main limit for large rasters, and our pilot test saw a $\sim 3\times$ difference between uncached and cached runs at 5 m. With a verified cold cache and a fast NVMe SSD, we instead find `gdaldem` to be compute-bound ($\sim$73%) and the cache state to be irrelevant ($\leq$2%). The pilot timed runs through the QGIS interface on a different laptop, so its first runs likely included one-off start-up and loading costs. For display, our results agree with the pyramid-based approaches of Losasso and Hoppe [6] and Campos et al. [7].

Implications. (1) The largest gain for processing should come from parallel computation across the 22 threads; decompression must also be parallel, since reading and decompressing the 1 m grid alone takes 29 s on one thread while the SSD could deliver it in about 4–5 s. (2) Computing several derivatives in one pass avoids re-reading and re-writing the grid ($\sim$27% of each `gdaldem` run). (3) On-demand computation is cheap: slope for one 256×256 tile costs $\sim$1.6 ms on one thread, so an interactive viewer is limited by reading the right window at the right pyramid level.

Validity. Repeats agree closely: the median IQR per configuration is 1.4% (E1) and 1.3% (E2) of the median; medians absorb the few single slow runs. On the synthetic 2 m/1 m grids warm values are single runs. Sizes use MB = 10^6 and GB = 10^9 bytes. Runs below $\sim$10 million cells are dominated by program start-up and are not used for per-cell claims. Synthetic grids are used only for timing. Earlier, disturbed attempts (before pinning and before handling background scanning) are kept as evidence.

6 Next steps

1. **Our engine (E3, E4):** tiled processing with overlapping borders (halos), NumPy reference, Numba parallel CPU and CuPy GPU kernels for the same six operators, computing several derivatives in one pass; validated against `gdaldem`; measured against the baseline above, including CPU–GPU transfer times.
2. **Pyramid on demand (E5, gap G2):** compute slope directly from each pyramid level and measure its error against the real 2 m slope, together with the time saved, split by slope class.
3. **Interactive viewer (E6):** a tile server computing slope/aspect/hillshade tiles on the fly for QGIS and a Cesium globe; tile latency for precomputed vs on-the-fly CPU vs GPU.
4. **Stretch:** viewshed (R3 vs XDraw vs `gdal_viewshed`) and terrain distance (Dijkstra vs A*).

References

[1] European Space Agency (2021). Copernicus Global Digital Elevation Model (GLO-30). Distributed by OpenTopography. doi:10.5069/G9028PQB.

[2] Shean, D. (2017). High Mountain Asia 8-meter DEM Mosaics Derived from Optical Imagery, Version 1. NASA NSIDC DAAC. doi:10.5067/KXOVQ9L172S2.

[3] Bhushan, S., & Shean, D. (2021). Chamoli Disaster Pre-event 2-m DEM Composite: September 2015. Zenodo. doi:10.5281/zenodo.4554647.

[4] Horn, B. K. P. (1981). Hill shading and the reflectance map. *Proceedings of the IEEE*, 69(1), 14–47.

[5] Zevenbergen, L. W., & Thorne, C. R. (1987). Quantitative analysis of land surface topography. *Earth Surface Processes and Landforms*, 12(1), 47–56.

[6] Losasso, F., & Hoppe, H. (2004). Geometry clipmaps: Terrain rendering using nested regular grids. *ACM Transactions on Graphics*, 23(3), 769–776.

[7] Campos, R., et al. (2020). 3D simplification methods and large scale terrain tiling. *Remote Sensing*, 12(3), 437.